final report

Title of RDBMS IA2:

Group members <Rollno, Name> :
Aarushi Gosalia - 16010124096
Riya Gupta - 16010124100
Ashwera Hasan - 16010124107

Objective of the Implementation :
This implementation will be aimed at comparing the efficiency of various selection algorithms that can be employed by database management systems. In particular, the objective of this implementation is to develop and test the execution time of various algorithms such as linear scan, sorting and binary search, and indexing for performing selections in database systems for simple and complicated queries.

Theory/Algorithm/s used:
The following method is based on three selection methods:

1. Linear Scan (Tuple-Based Selection)
   Scanning takes place sequentially for each row (tuple) in the database.
   The condition (e.g., salary > threshold) is examined for each tuple.
   Complexity:
   - Best case: O(n)
   - Worst case: O(n)

   Good for small databases but poor for large databases.
2. Sort-Based Selection
   Dataset is first sorted according to the selection key (salary).
   Binary search is performed to find the boundary that satisfies the condition.
   The remaining elements are counted.
   Time complexity:
   - Sorting: O(n log n)
   - Searching: O(log n)

   Suitable for multiple searches after sorting.
3. Index-Based Selection
   The index (using the map, equivalent to B-tree) is constructed.
   The maps assign value (salary) to records.
   The query uses upper_bound() to retrieve data.

Time Complexity:

- Constructing Index: O(n log n)
- Retrieving Data: O(log n + k) (k = number of matching records)

Optimal for repetitive queries due to maps since now repetitive scanning is not required.

4. Complex Selection
   Multiple conditions such as salary > x AND department = "IT" are used.
   The program utilizes tuple-oriented technique.
   Shows that there is an increase in time cost due to the check of multiple conditions.

Dataset / Relational Schema/Input used:
Employee Relation

- empID (PK)
- name (String)
- department (String)
- salary (Integer)
- age (Integer)

Code <with comments> :

```
#include <bits/stdc++.h>

using namespace std;

using namespace chrono; // to measure time taken by the algo



// Employee DB structure

struct Employee {
```

```
    int empID;
    string name;
    string department;
    int salary;
    int age;
};



// Generate random dataset of size n
vector<Employee> generateData(int n) {
    vector<Employee> data;
    string departments[] = {"HR", "IT", "Sales", "Finance"};
    for (int i = 0; i < n; i++) {
        Employee e;
        e.empID = i + 1;
        e.name = "Emp" + to_string(i + 1);
        e.department = departments[rand() % 4];
        e.salary = rand() % 100000; // Range of salary taken from 0
to 99999
        e.age = 20 + rand() % 40;   // Range of age taken from 20
to 59
        data.push_back(e);
    }
    return data;
}
```

```
// Tuple Based Selection (Linear Scan)
int tupleSelection(vector<Employee>& data, int threshold) {
    int count = 0;
    for (auto& e : data) {
        if (e.salary > threshold)
            count++; //maintains count of employees with salary
more than threshold.
    }
    return count;
}


// Sort Based Selection
//Sorted array can help implement binary search
int sortSelection(vector<Employee> data, int threshold) {
    //Sort all tuples by salary
    sort(data.begin(), data.end(), [](const Employee& a, const
Employee& b) {
        return a.salary < b.salary;
    });

    // Binary search for first salary > threshold
    int left = 0, right = (int)data.size() - 1;
```

```
    int idx = (int)data.size(); // default = no match found
    while (left <= right) {
        int mid = (left + right) / 2;
        if (data[mid].salary > threshold) {
            idx = mid;        // potential boundary, go left
            right = mid - 1;
        } else {
            left = mid + 1;   // go right
        }
    }

    // All records from idx onward satisfy condition
    return (int)data.size() - idx;
}

// Index based selection
/* using map to create an index on salary for faster data retrieval
k = number of matching records */

// Build index
// maps salary value to the list of empIDs with that salary
map<int, vector<int>> buildIndex(vector<Employee>& data) {
    map<int, vector<int>> index;
    for (auto& e : data)
```

```
        index[e.salary].push_back(e.empID);
    return index;
}

// Query index
// upper_bound() jumps directly to first key > threshold in log n time
int indexSelection(map<int, vector<int>>& index, int threshold) {
    int count = 0;
    for (auto it = index.upper_bound(threshold); it != index.end(); it++)
        count += it->second.size();
    return count;
}

// Two conditions combined using linear scan

int complexTupleSelection(vector<Employee>& data) {
    int count = 0;
    for (auto& e : data)
        if (e.salary > 50000 && e.department == "IT")
            count++;
    return count;
}
```

```
// Tests all algorithms on 4 dataset sizes
// Time measured in microseconds

int main() {
    srand(42); // fixed seed ensures same dataset every run

    vector<int> sizes = {100, 1000, 50000, 100000}; // to check
every algorithms on different sizes

    for (int n : sizes) {
        cout << "\nDataset Size: " << n << endl;

        vector<Employee> data = generateData(n);

        // Tuple-based timing
        auto t1 = high_resolution_clock::now();
        int r1 = tupleSelection(data, 50000);
        auto t2 = high_resolution_clock::now();
        cout << "Tuple-based: " << duration_cast<microseconds>(t2 -
t1).count() << " us" << endl;

        // Sort-based timing (includes sort cost)
        t1 = high_resolution_clock::now();
        int r2 = sortSelection(data, 50000);
```

```
        t2 = high_resolution_clock::now();

        cout << "Sort-based: " << duration_cast<microseconds>(t2 -
t1).count() << " us" << endl;



        // Index build timing

        t1 = high_resolution_clock::now();

        auto index = buildIndex(data);

        t2 = high_resolution_clock::now();

        int buildTime = duration_cast<microseconds>(t2 -
t1).count();

        cout << "Index Build: " << buildTime << " us" << endl;



        //Index query timing (after index is ready)

        t1 = high_resolution_clock::now();

        int r3 = indexSelection(index, 50000);

        t2 = high_resolution_clock::now();

        int queryTime = duration_cast<microseconds>(t2 -
t1).count();

        cout << "Index Query: " << queryTime << " us" << endl;

        cout << "Index Total: " << (buildTime + queryTime) << " us"
<< endl;



        // Complex selection timing

        t1 = high_resolution_clock::now();

        int r4 = complexTupleSelection(data);

        t2 = high_resolution_clock::now();
```

```
        cout << "Complex Select: " <<
duration_cast<microseconds>(t2 - t1).count() << " us" << endl;



    }



    return 0;

}
```

Results<snapshots>:

```
PS C:\Users\RIYA\Desktop\study\SY\rdbms\ia2> g++ code.cpp -o code.exe
PS C:\Users\RIYA\Desktop\study\SY\rdbms\ia2> .\code.exe

Dataset Size: 100
Tuple-based: 1 us
Sort-based: 316 us
Index Build: 194 us
Index Query: 1 us
Index Total: 195 us
Complex Select: 2 us

Dataset Size: 1000
Tuple-based: 3 us
Sort-based: 3492 us
Index Build: 1570 us
Index Query: 3 us
Index Total: 1573 us
Complex Select: 10 us

Dataset Size: 50000
Tuple-based: 627 us
Sort-based: 243474 us
Index Build: 83710 us
Index Query: 2 us
Index Total: 83712 us
Complex Select: 551 us

Dataset Size: 100000
Tuple-based: 1089 us
Sort-based: 578279 us
Index Build: 169445 us
Index Query: 2 us
Index Total: 169447 us
Complex Select: 1040 us
```

Analysis of results:

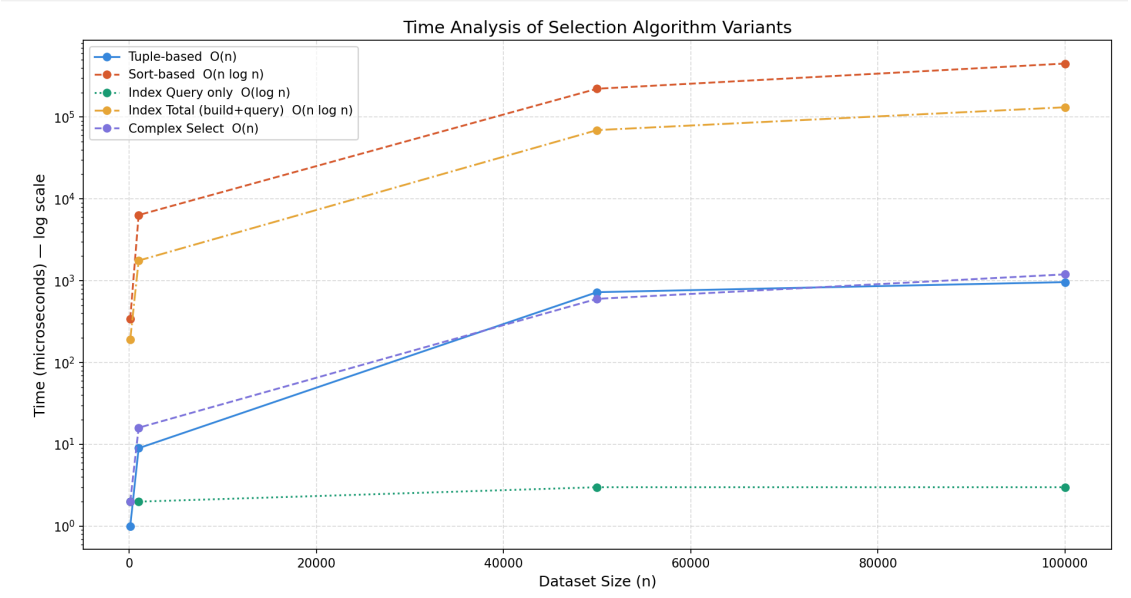
The performance of three selection methods, tuple-based, sort-based, and index-based, was analyzed for the number of records 100, 50,000, and 100,000.

- **Tuple-Based Selection (Linear Scan):**
  Execution time had a linear correlation with the number of records, implying an O(n) time complexity. There was little difference in the performance for

smaller data sets. But with large data sets, there was a considerable increase in execution time as the entire data set was traversed.

- **Sort-Based Selection:**
  The time taken to perform operations was high because of the initial cost of sorting the list of records (O(n log n)), but thereafter, selection was fast owing to the binary search performed (O(log n)). This technique would prove advantageous if multiple selection operations were to be performed on the data set.
- **Index-Based Selection:**
  Indexing approach took more time in index creation but returned the fastest query result execution. With the use of map (B-tree structure) it did not scan all the data, only accessing the necessary entries. Although the overall time (index creation + query result execution) was faster than other approaches when the dataset was very large.
- **Complex Selection Query:**
  The complex query (salary > 50000 AND department = "IT") with the tuple-based approach for selection took more time to execute than the simple ones because of more conditions.
- **General Observation:**
  The larger the size of the database, the higher the difference between the results of performance for the various algorithms.

Graph comparing Time Complexity :



Conclusion:

The above experiment clearly indicates that different kinds of selection algorithms perform differently based on the size of the dataset and queries.

- **The tuple scan selection** algorithm works best for smaller datasets, while it suffers from poor scalability because of its time complexity of O(N).
- **The sort scan technique** offers better performance in repeated queries, although there is an additional initial overhead in sorting.
- **The index selection** method performs best compared to others, making it one of the most common techniques in database indexing.

Overall, the results highlight the importance of choosing appropriate selection techniques in database systems to optimize query performance. Indexing plays a crucial role in improving efficiency and is widely used in real-world database management systems.

References:

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill.
2. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
4. cppreference.com contributors. (n.d.). *C++ standard library reference*. Retrieved April 20, 2026, from https://en.cppreference.com
5. Lecture notes and course material on Relational Database Management Systems (RDBMS) and Analysis of Algorithms

Plagiarism Report of the code
//reason intimated in person

content

given topic -> Implement and perform time analysis on the following variants of select algorithms (for simple selection or complex selection) :
Tuple-based or Sort-based , Index-based
Compare the performance of different implementations with different sizes of data sets.

Topic -> time analysis of selection algorithm variants; tuple based, sort-based, index - based

generated an employee dataset of various size : 100, 50000, 100000

used linear search for tuple based

sorted then binary search for sort based

used a map to store emp id vectors for a particular salary, then searched

for index based -> build time to make the map (b-tree like sorted) + accessing the reqd ids time; but its the cheapest since we build only once & search doesnt require us iterating thru the entire array

above all were simple select queries when salary > x

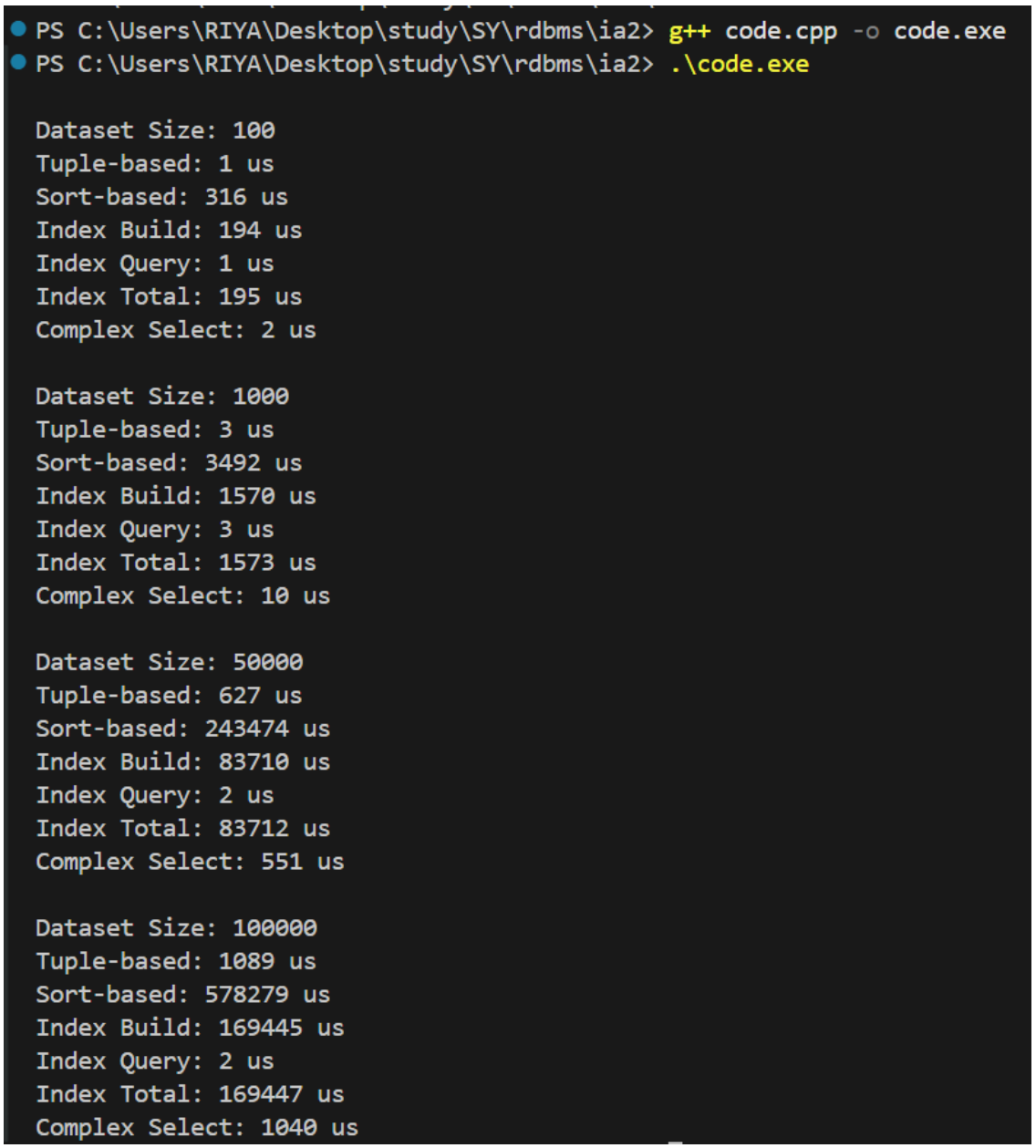
complex select added w tuple based where salary > x and dept = “abc”

printed time in microseconds since ms couldnt show difference, it was all under 1ms

[can use more dataset sizes + do complex select w more]

code :

Output :



Objective of the Implementation

This implementation will be aimed at comparing the efficiency of various selection algorithms that can be employed by database management systems. In particular, the objective of this implementation is to develop and test the execution time of various algorithms such as linear scan, sorting and binary search, and indexing for performing selections in database systems for simple and complicated queries.

Theory / Algorithms Used

The following method is based on three selection methods:

1. Linear Scan (Tuple-Based Selection)

Scanning takes place sequentially for each row (tuple) in the database.

The condition (e.g., salary > threshold) is examined for each tuple.

Complexity:

- Best case: O(n)
- Worst case: O(n)

Good for small databases but poor for large databases.

2. Sort-Based Selection

Dataset is first sorted according to the selection key (salary).

Binary search is performed to find the boundary that satisfies the condition.

The remaining elements are counted.

Time complexity:

- Sorting: O(n log n)
- Searching: O(log n)

Suitable for multiple searches after sorting.

3. Index-Based Selection

The index (using the map, equivalent to B-tree) is constructed.

The maps assign value (salary) to records.

The query uses upper_bound() to retrieve data.

Time Complexity:

- Constructing Index: O(n log n)
- Retrieving Data: O(log n + k) (k = number of matching records)

Optimal for repetitive queries, as scanning is not required.

4. Complex Selection

- Conditions such as salary > x AND department = “IT” are used.
- The program utilizes tuple-oriented technique.
- Shows that there is an increase in cost due to the check of multiple conditions.

Dataset / Relational Schema / Input Used

Employee Relation

- empID (PK)
- name (String)
- department (String)
- salary (Integer)
- age (Integer)

Dataset Details:

Artificial data created using a program.

Sizes employed during experiment:

- 100 items
- 50,000 items
- 100,000 items

Dataset Features:

- Departments: HR, IT, Sales, Finance
- Salary: Random numbers (0 – 99,999)
- Age: 20 – 59
- Employee IDs unique

Queries Applied As Input:

Selection Query:

- salary > 50000

Compound Selection Query:

- salary > 50000 AND department = "IT"

Evaluation Metrics:

Measurement of time taken using high_resolution_clock

Time units: microseconds (µs)

Parameters to measure:

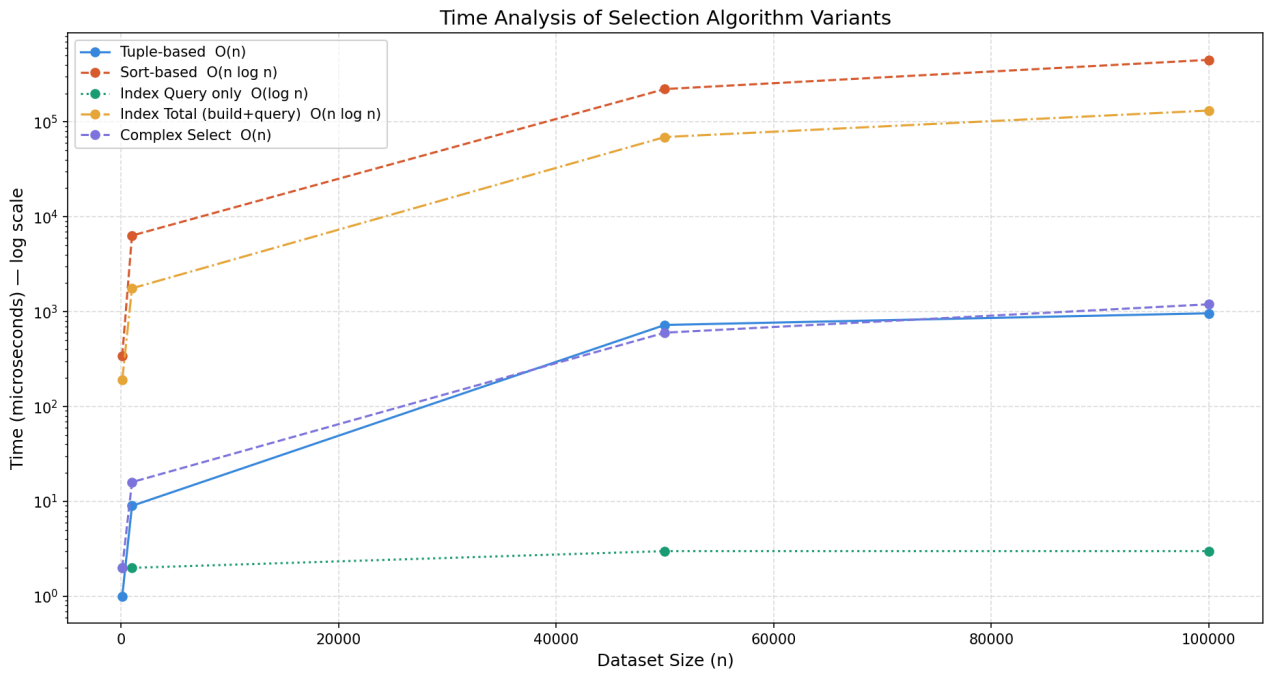
- Time taken by algorithms to execute
- Query + index creation time

Analysis of Results

The performance of three selection methods, tuple-based, sort-based, and index-based, was analyzed for the number of records 100, 50,000, and 100,000.

- **Tuple-Based Selection (Linear Scan):**
  Execution time had a linear correlation with the number of records, implying an O(n) time complexity. There was little difference in the performance for smaller data sets. But with large data sets, there was a considerable increase in execution time as the entire data set was traversed.
- **Sort-Based Selection:**
  The time taken to perform operations was high because of the initial cost of sorting the list of records (O(n log n)), but thereafter, selection was fast owing to the binary search performed (O(log n)). This technique would prove advantageous if multiple selection operations were to be performed on the data set.
- **Index-Based Selection:**
  Indexing approach took more time in index creation but returned the fastest query result execution. With the use of map (B-tree structure) it did not scan all the data, only accessing the necessary entries. Although the overall time (index creation + query result execution) was faster than other approaches when the dataset was very large.
- **Complex Selection Query:**
  The complex query (salary > 50000 AND department = "IT") with the tuple-based approach for selection took more time to execute than the simple ones because of more conditions.
- **General Observation:**
  The larger the size of the database, the higher the difference between the results of performance for the various algorithms.

Graph comparing Time Complexity :



Conclusion

The above experiment clearly indicates that different kinds of selection algorithms perform differently based on the size of the dataset and queries.

- **The tuple scan selection** algorithm works best for smaller datasets, while it suffers from poor scalability because of its time complexity of O(N).
- **The sort scan technique** offers better performance in repeated queries, although there is an additional initial overhead in sorting.
- **The index selection** method performs best compared to others, making it one of the most common techniques in database indexing.

Overall, the results highlight the importance of choosing appropriate selection techniques in database systems to optimize query performance. Indexing plays a crucial role in improving efficiency and is widely used in real-world database management systems.

References

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database system concepts* (7th ed.). McGraw-Hill.

2. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.
4. cppreference.com contributors. (n.d.). *C++ standard library reference*. Retrieved April 20, 2026, from https://en.cppreference.com
5. Lecture notes and course material on Relational Database Management Systems (RDBMS) and Analysis of Algorithms

Tab 3

Code <with comments> :

```cpp
#include <bits/stdc++.h>

using namespace std;

using namespace chrono; // to measure time taken by the algo



// Employee DB structure

struct Employee {

    int empID;

    string name;

    string department;

    int salary;

    int age;

};




// Generate random dataset of size n

vector<Employee> dataset(int n) {

    vector<Employee> data;

    string departments[] = {"HR", "IT", "Sales", "Finance"};

    for (int i = 0; i < n; i++) {

        Employee e;

        e.empID = i + 1;

        e.name = "Emp" + to_string(i + 1);

        e.department = departments[rand() % 4];

        e.salary = rand() % 100000; // Range of salary taken from 0
to 99999
```

```
        e.age = 20 + rand() % 40;   // Range of age taken from 20
to 59

        data.push_back(e);

    }

    return data;

}




// Tuple Based Selection (Linear Scan)

int tuplebased(vector<Employee>& data, int condn) {

    int count = 0;

    for (auto& e : data) {

        if (e.salary > condn)

            count++; //counts employees with salary more than a
particular number

    }

    return count;

}




// Sort Based Selection

//Sorted array can help implement binary search

int sortbased(vector<Employee> data, int condn) {

    //Sort all tuples by salary

    sort(data.begin(), data.end(), [](const Employee& a, const
Employee& b) {

        return a.salary < b.salary;

    });
```

```
    // Binary search for first salary > threshold
    int l = 0, r = (int)data.size() - 1;
    int idx = (int)data.size(); // default = no match found
    while (l <= r) {
        int mid = (l + r) / 2;
        if (data[mid].salary > condn) {
            idx = mid;       // potential boundary, go left
            r = mid - 1;
        } else {
            l = mid + 1;  // go right
        }
    }

    // All records from idx onward satisfy condition
    return (int)data.size() - idx;
}

// Index based selection
/* using map to create an index on salary for faster data retrieval
k = number of matching records */

// Build index
// maps salary value to the list of empIDs with that salary
map<int, vector<int>> buildIndex(vector<Employee>& data) {
    map<int, vector<int>> index;
    for (auto& e : data)
```

```
        index[e.salary].push_back(e.empID);
    return index;
}

// Query index
// upper_bound() jumps directly to first key > threshold in log n time
int indexbased(map<int, vector<int>>& index, int condn) {
    int count = 0;
    for (auto it = index.upper_bound(condn); it != index.end(); it++)
        count += it->second.size();
    return count;
}

// Two conditions combined using linear scan

int complexselect(vector<Employee>& data) {
    int count = 0;
    for (auto& e : data)
        if (e.salary > 50000 && e.department == "IT")
            count++;
    return count;
}

// Tests all algorithms on 4 dataset sizes
// Time measured in microseconds
```

```cpp
int main() {
    srand(42); // fixed seed ensures same dataset every run

    vector<int> sizes = {100, 1000, 50000, 100000}; // to check every algorithms on different sizes

    for (int n : sizes) {
        cout << "\nDataset Size: " << n << endl;

        vector<Employee> data = generateData(n);

        // Tuple-based timing
        auto t1 = high_resolution_clock::now();
        int r1 = tupleSelection(data, 50000);
        auto t2 = high_resolution_clock::now();
        cout << "Tuple-based: " << duration_cast<microseconds>(t2 - t1).count() << " us" << endl;

        // Sort-based timing (includes sort cost)
        t1 = high_resolution_clock::now();
        int r2 = sortSelection(data, 50000);
        t2 = high_resolution_clock::now();
        cout << "Sort-based: " << duration_cast<microseconds>(t2 - t1).count() << " us" << endl;

        // Index build timing
        t1 = high_resolution_clock::now();
```

```
        auto index = buildIndex(data);

        t2 = high_resolution_clock::now();

        int buildTime = duration_cast<microseconds>(t2 -
t1).count();

        cout << "Index Build: " << buildTime << " us" << endl;



        //Index query timing (after index is ready)

        t1 = high_resolution_clock::now();

        int r3 = indexSelection(index, 50000);

        t2 = high_resolution_clock::now();

        int queryTime = duration_cast<microseconds>(t2 -
t1).count();

        cout << "Index Query: " << queryTime << " us" << endl;

        cout << "Index Total: " << (buildTime + queryTime) << " us"
<< endl;



        // Complex selection timing

        t1 = high_resolution_clock::now();

        int r4 = complexTupleSelection(data);

        t2 = high_resolution_clock::now();

        cout << "Complex Select: " <<
duration_cast<microseconds>(t2 - t1).count() << " us" << endl;



    }



    return 0;
```

file

This implementation will be aimed at comparing the efficiency of various selection algorithms that can be employed by database management systems. In particular, the objective of this implementation is to develop and test the execution time of various algorithms such as linear scan, sorting and binary search, and indexing for performing selections in database systems for simple and complicated queries.

The following method is based on three selection methods:

1. Linear Scan (Tuple-Based Selection)
   Scanning takes place sequentially for each row (tuple) in the database.
   The condition (e.g., salary > threshold) is examined for each tuple.
   Complexity:
   - Best case: O(n)
   - Worst case: O(n)

   Good for small databases but poor for large databases.
2. Sort-Based Selection
   Dataset is first sorted according to the selection key (salary).
   Binary search is performed to find the boundary that satisfies the condition.
   The remaining elements are counted.
   Time complexity:
   - Sorting: O(n log n)
   - Searching: O(log n)

   Suitable for multiple searches after sorting.
3. Index-Based Selection
   The index (using the map, equivalent to B-tree) is constructed.
   The maps assign value (salary) to records.
   The query uses upper_bound() to retrieve data.
   Time Complexity:
   - Constructing Index: O(n log n)
   - Retrieving Data: O(log n + k) (k = number of matching records)

   Optimal for repetitive queries due to maps since now repetitive scanning is not required.
4. Complex Selection
   Multiple conditions such as salary > x AND department = “IT” are used.
   The program utilizes tuple-oriented technique.
   Shows that there is an increase in time cost due to the check of multiple conditions.

Employee Relation

- empID (PK)
- name (String)

- department (String)
- salary (Integer)
- age (Integer)


```cpp
#include <bits/stdc++.h>

using namespace std;

using namespace chrono; // to measure time taken by the algo



// Employee DB structure

struct Employee {

    int empID;

    string name;

    string department;

    int salary;

    int age;

};




// Generate random dataset of size n

vector<Employee> generateData(int n) {

    vector<Employee> data;

    string departments[] = {"HR", "IT", "Sales", "Finance"};

    for (int i = 0; i < n; i++) {
```

```
        Employee e;
        e.empID = i + 1;
        e.name = "Emp" + to_string(i + 1);
        e.department = departments[rand() % 4];
        e.salary = rand() % 100000; // Range of salary taken from 0
to 99999
        e.age = 20 + rand() % 40;   // Range of age taken from 20
to 59
        data.push_back(e);
    }
    return data;
}



// Tuple Based Selection (Linear Scan)
int tupleSelection(vector<Employee>& data, int threshold) {
    int count = 0;
    for (auto& e : data) {
        if (e.salary > threshold)
            count++; //maintains count of employees with salary
more than threshold.
    }
    return count;
}



// Sort Based Selection
//Sorted array can help implement binary search
```

```
int sortSelection(vector<Employee> data, int threshold) {

    //Sort all tuples by salary

    sort(data.begin(), data.end(), [](const Employee& a, const
Employee& b) {

        return a.salary < b.salary;

    });



    // Binary search for first salary > threshold

    int left = 0, right = (int)data.size() - 1;

    int idx = (int)data.size(); // default = no match found

    while (left <= right) {

        int mid = (left + right) / 2;

        if (data[mid].salary > threshold) {

            idx = mid;       // potential boundary, go left

            right = mid - 1;

        } else {

            left = mid + 1;  // go right

        }

    }



    // All records from idx onward satisfy condition

    return (int)data.size() - idx;

}



// Index based selection

/* using map to create an index on salary for faster data retrieval

k = number of matching records */
```

```
// Build index
// maps salary value to the list of empIDs with that salary
map<int, vector<int>> buildIndex(vector<Employee>& data) {
    map<int, vector<int>> index;
    for (auto& e : data)
        index[e.salary].push_back(e.empID);
    return index;
}

// Query index
// upper_bound() jumps directly to first key > threshold in log n time
int indexSelection(map<int, vector<int>>& index, int threshold) {
    int count = 0;
    for (auto it = index.upper_bound(threshold); it != index.end(); it++)
        count += it->second.size();
    return count;
}

// Two conditions combined using linear scan

int complexTupleSelection(vector<Employee>& data) {
    int count = 0;
    for (auto& e : data)
        if (e.salary > 50000 && e.department == "IT")
            count++;
```

```
    return count;
}



// Tests all algorithms on 4 dataset sizes
// Time measured in microseconds

int main() {
    srand(42); // fixed seed ensures same dataset every run

    vector<int> sizes = {100, 1000, 50000, 100000}; // to check
every algorithms on different sizes

    for (int n : sizes) {
        cout << "\nDataset Size: " << n << endl;

        vector<Employee> data = generateData(n);

        // Tuple-based timing
        auto t1 = high_resolution_clock::now();
        int r1 = tupleSelection(data, 50000);
        auto t2 = high_resolution_clock::now();
        cout << "Tuple-based: " << duration_cast<microseconds>(t2 -
t1).count() << " us" << endl;

        // Sort-based timing (includes sort cost)
        t1 = high_resolution_clock::now();
        int r2 = sortSelection(data, 50000);
```

```
        t2 = high_resolution_clock::now();

        cout << "Sort-based: " << duration_cast<microseconds>(t2 -
t1).count() << " us" << endl;



        // Index build timing

        t1 = high_resolution_clock::now();

        auto index = buildIndex(data);

        t2 = high_resolution_clock::now();

        int buildTime = duration_cast<microseconds>(t2 -
t1).count();

        cout << "Index Build: " << buildTime << " us" << endl;



        //Index query timing (after index is ready)

        t1 = high_resolution_clock::now();

        int r3 = indexSelection(index, 50000);

        t2 = high_resolution_clock::now();

        int queryTime = duration_cast<microseconds>(t2 -
t1).count();

        cout << "Index Query: " << queryTime << " us" << endl;

        cout << "Index Total: " << (buildTime + queryTime) << " us"
<< endl;



        // Complex selection timing

        t1 = high_resolution_clock::now();

        int r4 = complexTupleSelection(data);

        t2 = high_resolution_clock::now();

        cout << "Complex Select: " <<
duration_cast<microseconds>(t2 - t1).count() << " us" << endl;
```

```
    }


    return 0;

}
```

Results<snapshots>:

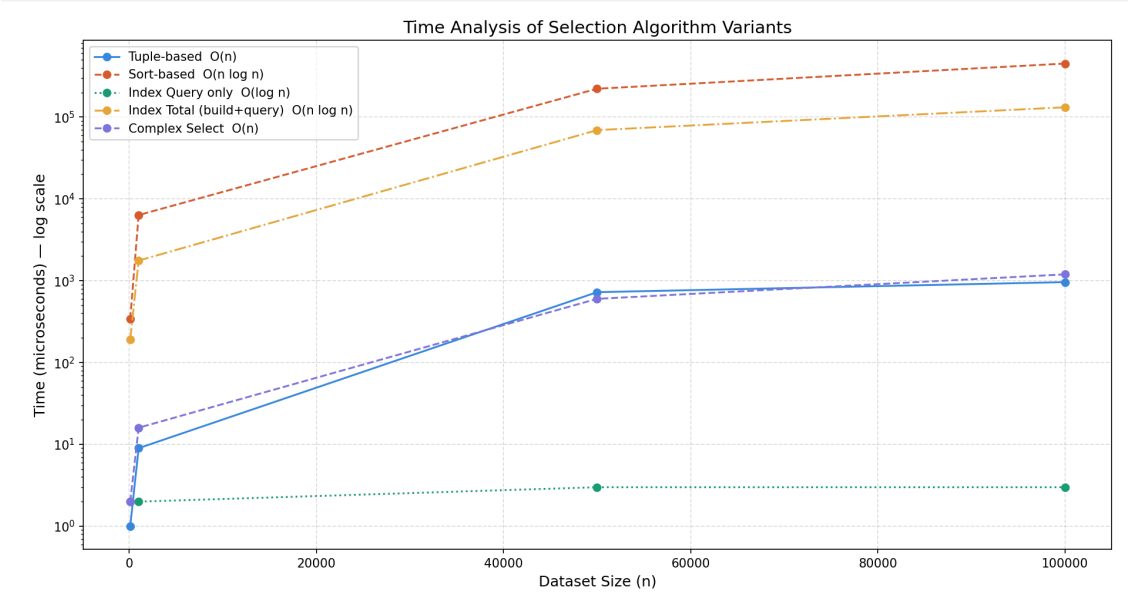
```
PS C:\Users\RIYA\Desktop\study\SY\rdbms\ia2> g++ code.cpp -o code.exe
PS C:\Users\RIYA\Desktop\study\SY\rdbms\ia2> .\code.exe

Dataset Size: 100
Tuple-based: 1 us
Sort-based: 316 us
Index Build: 194 us
Index Query: 1 us
Index Total: 195 us
Complex Select: 2 us

Dataset Size: 1000
Tuple-based: 3 us
Sort-based: 3492 us
Index Build: 1570 us
Index Query: 3 us
Index Total: 1573 us
Complex Select: 10 us

Dataset Size: 50000
Tuple-based: 627 us
Sort-based: 243474 us
Index Build: 83710 us
Index Query: 2 us
Index Total: 83712 us
Complex Select: 551 us

Dataset Size: 100000
Tuple-based: 1089 us
Sort-based: 578279 us
Index Build: 169445 us
Index Query: 2 us
Index Total: 169447 us
Complex Select: 1040 us
```

The performance of three selection methods, tuple-based, sort-based, and index-based, was analyzed for the number of records 100, 50,000, and 100,000.

- **Tuple-Based Selection (Linear Scan):**
  Execution time had a linear correlation with the number of records, implying an O(n) time complexity. There was little difference in the performance for

smaller data sets. But with large data sets, there was a considerable increase in execution time as the entire data set was traversed.

- **Sort-Based Selection:**
  The time taken to perform operations was high because of the initial cost of sorting the list of records (O(n log n)), but thereafter, selection was fast owing to the binary search performed (O(log n)). This technique would prove advantageous if multiple selection operations were to be performed on the data set.
- **Index-Based Selection:**
  Indexing approach took more time in index creation but returned the fastest query result execution. With the use of map (B-tree structure) it did not scan all the data, only accessing the necessary entries. Although the overall time (index creation + query result execution) was faster than other approaches when the dataset was very large.
- **Complex Selection Query:**
  The complex query (salary > 50000 AND department = "IT") with the tuple-based approach for selection took more time to execute than the simple ones because of more conditions.
- **General Observation:**
  The larger the size of the database, the higher the difference between the results of performance for the various algorithms.



Conclusion:

The above experiment clearly indicates that different kinds of selection algorithms perform differently based on the size of the dataset and queries.

- **The tuple scan selection** algorithm works best for smaller datasets, while it suffers from poor scalability because of its time complexity of O(N).
- **The sort scan technique** offers better performance in repeated queries, although there is an additional initial overhead in sorting.
- **The index selection** method performs best compared to others, making it one of the most common techniques in database indexing.

Overall, the results highlight the importance of choosing appropriate selection techniques in database systems to optimize query performance. Indexing plays a crucial role in improving efficiency and is widely used in real-world database management systems.